

Understanding AIS

part 3: AF interfaces

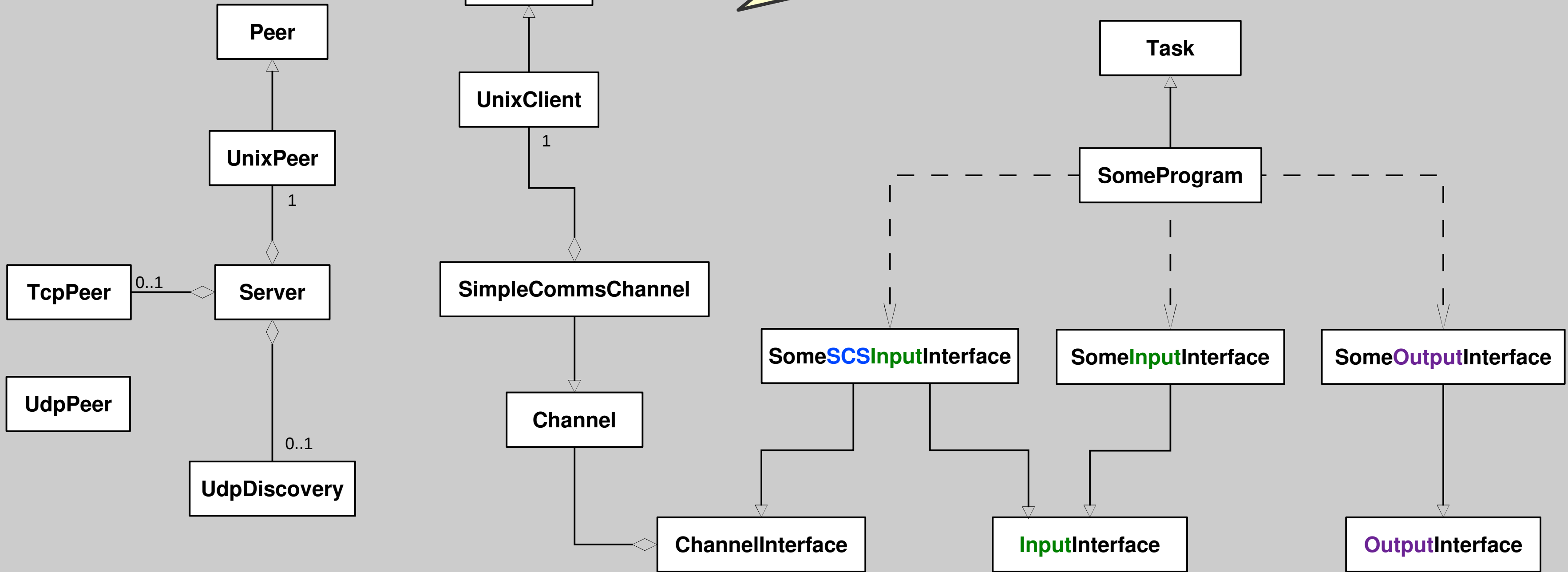
Introduction

- ◆ AIS stands for Autonomy Infrastructure Software
- ◆ AF stands for Application Framework
- ◆ Part 1 covers SCS
- ◆ Part 2 covers Autonomy Tasks
- ◆ Part 3 covers Input & Output interfaces in general
- ◆ Part 4 covers SCS interfaces

SCS + AF

ECU #1

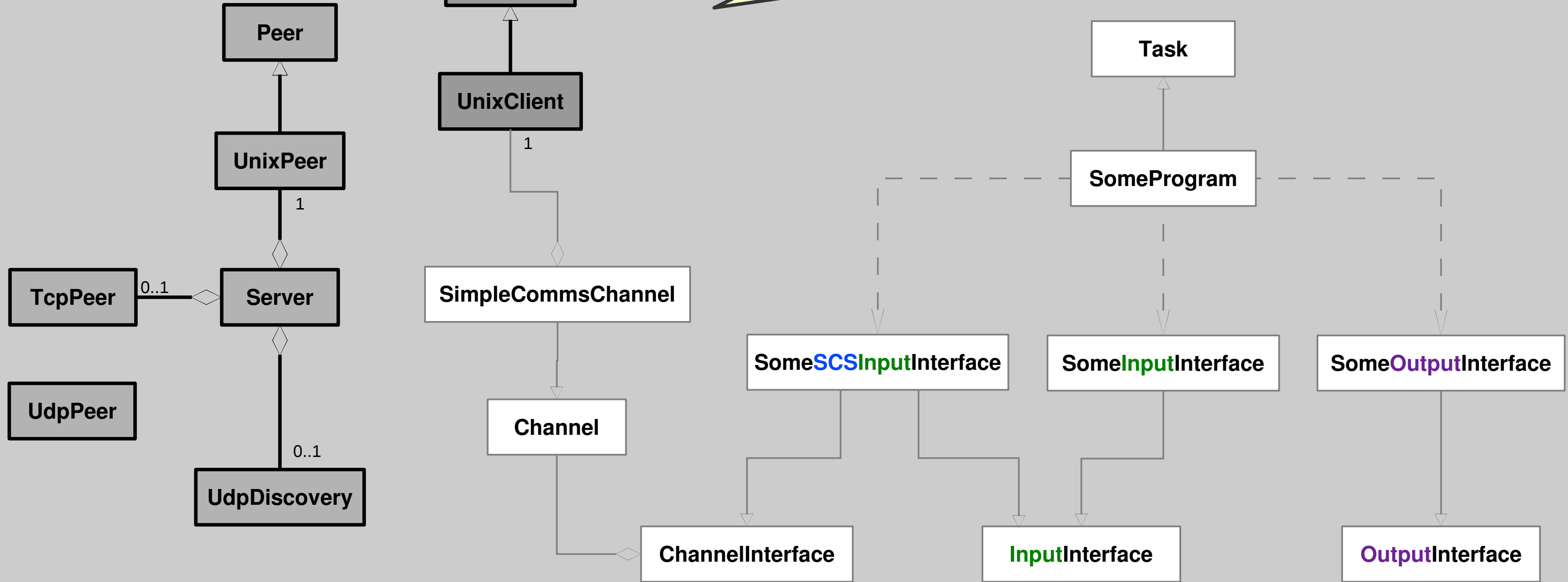
The high level view



scs server

ECU #1

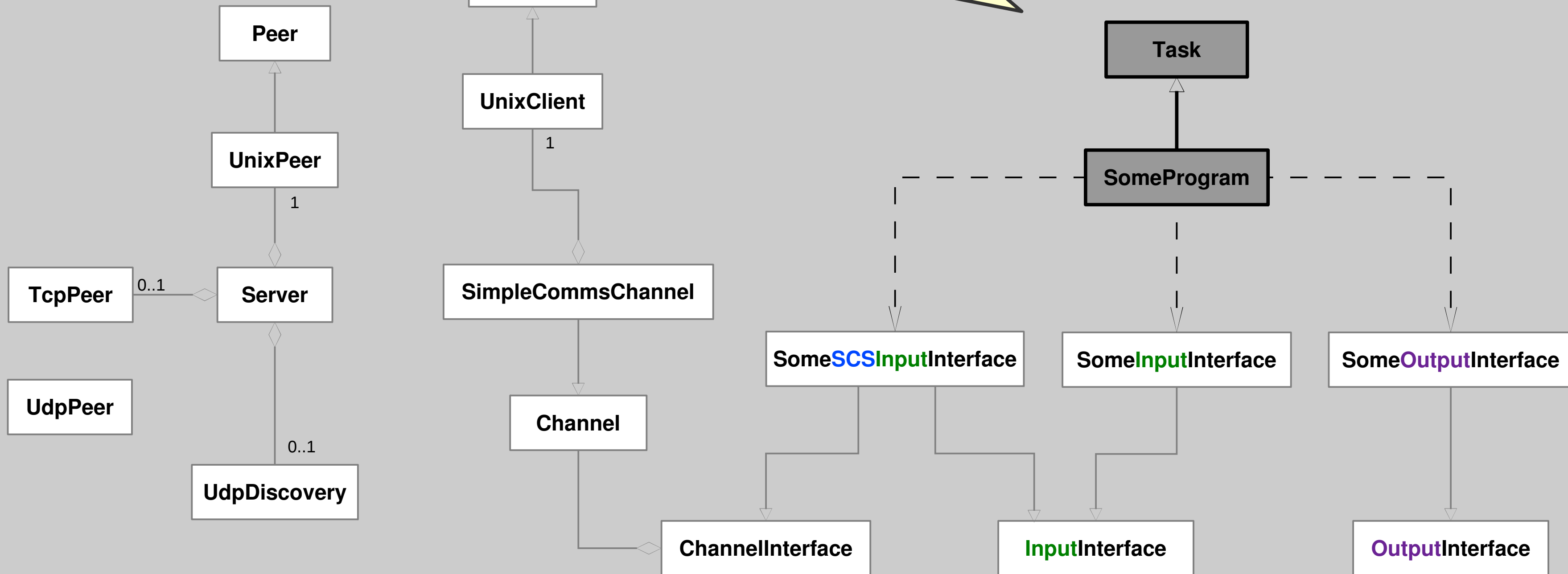
See AIS training part 1



Autonomy Task

ECU #1

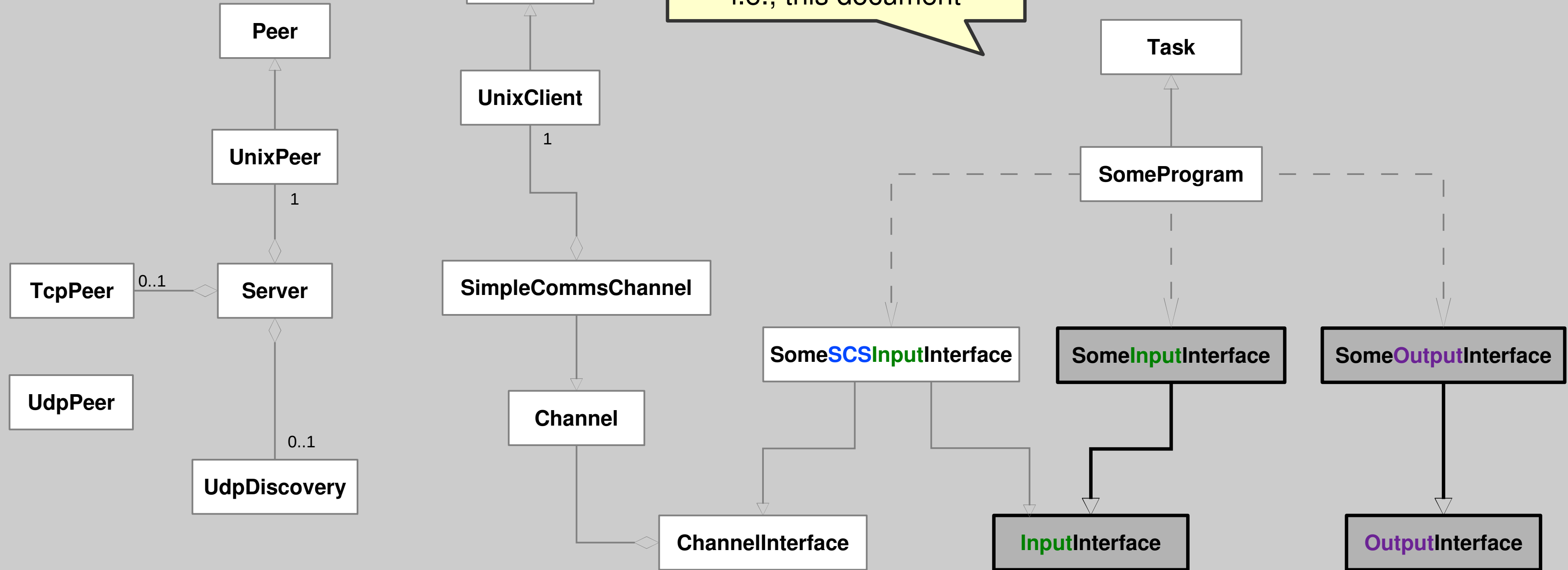
See AIS training part 2



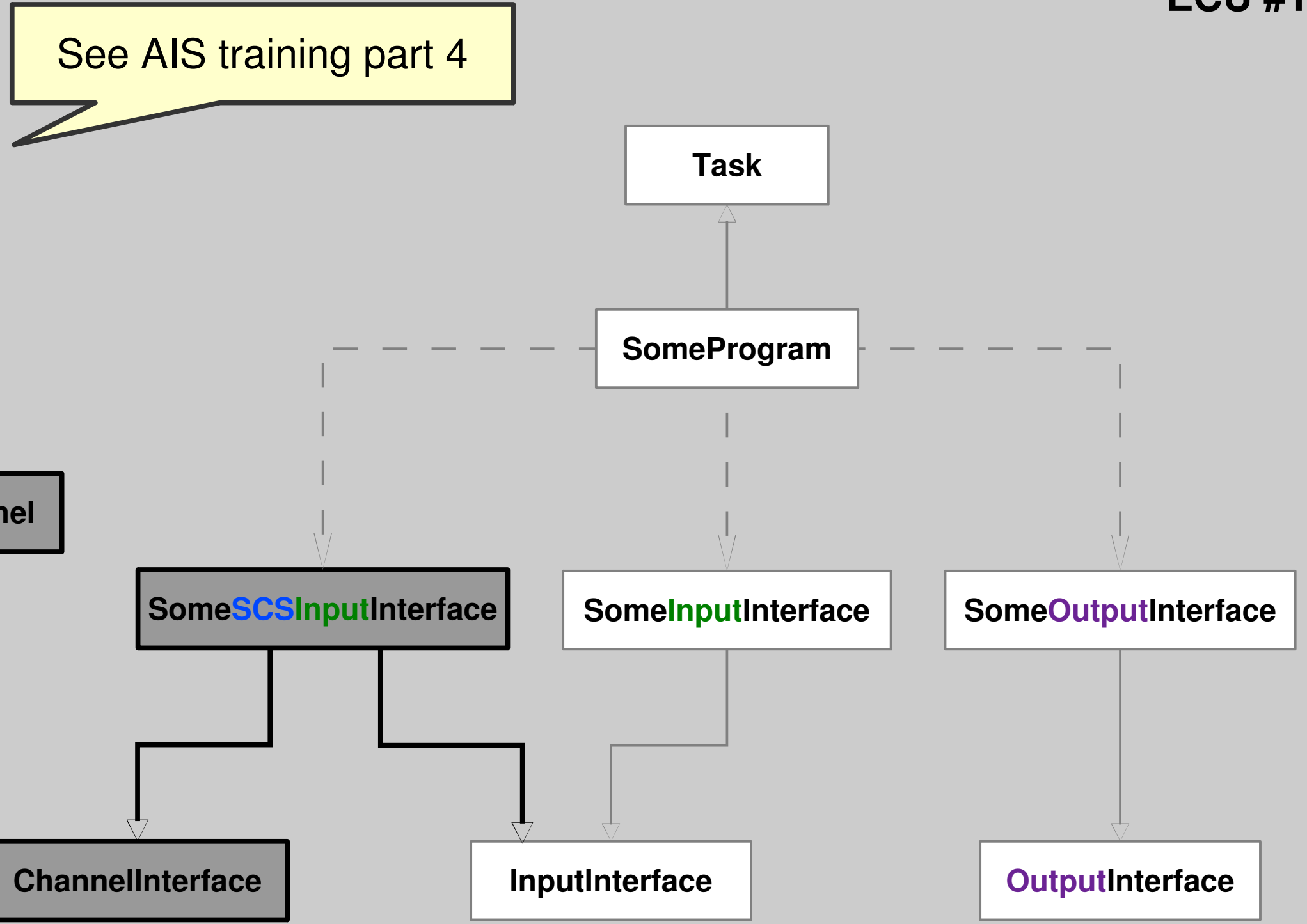
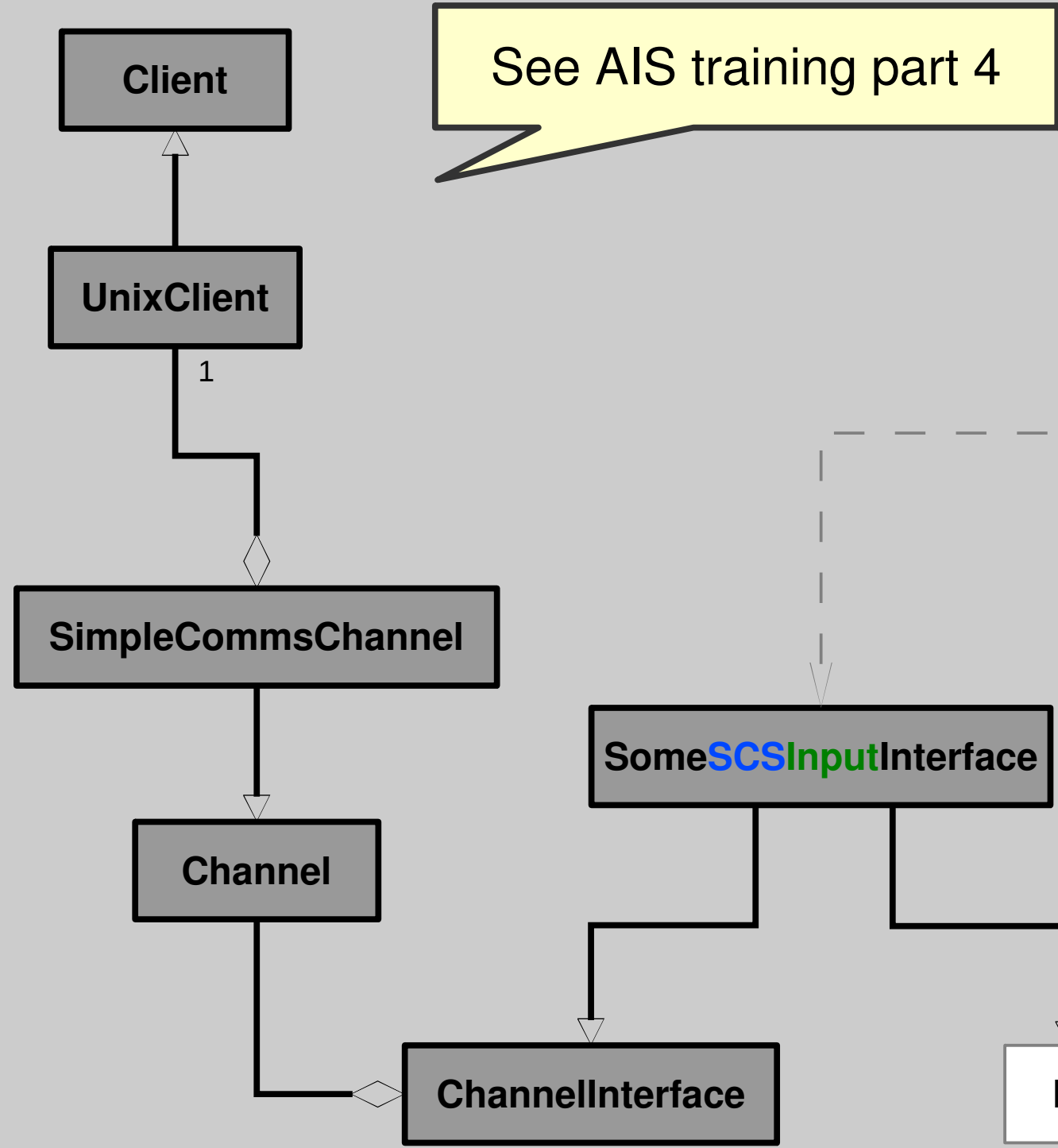
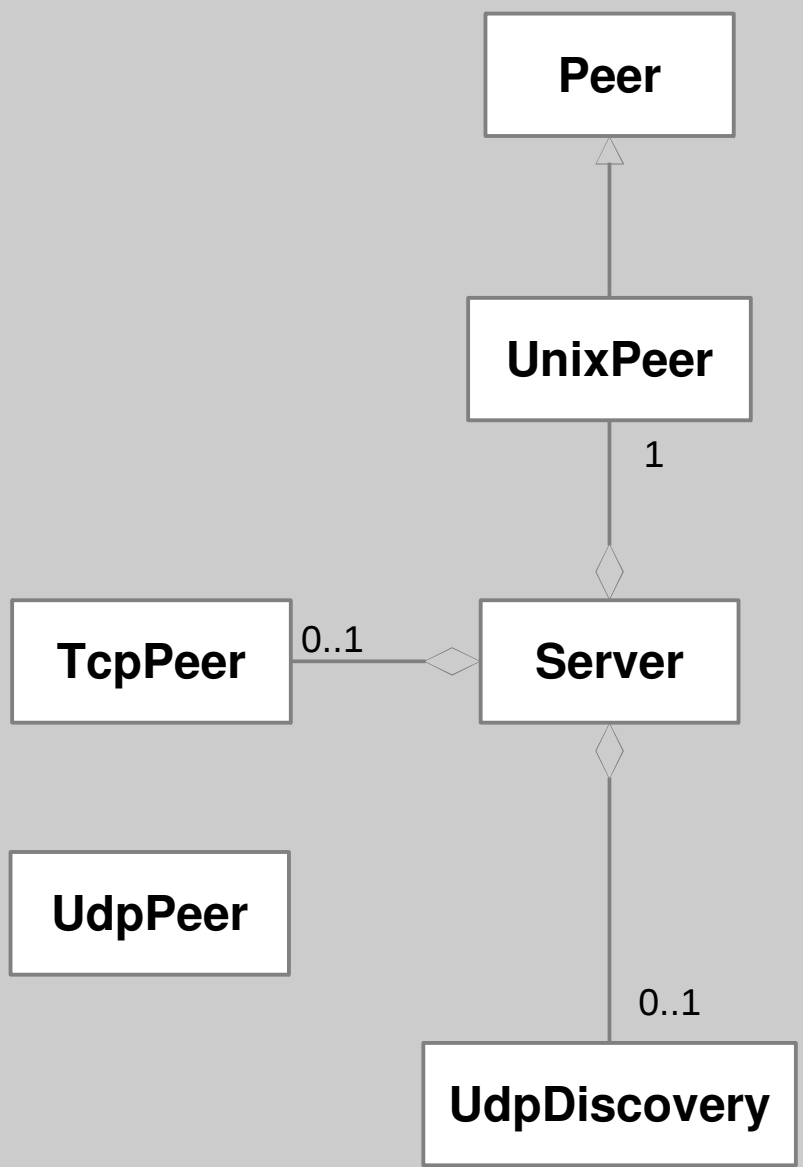
this document scope

ECU #1

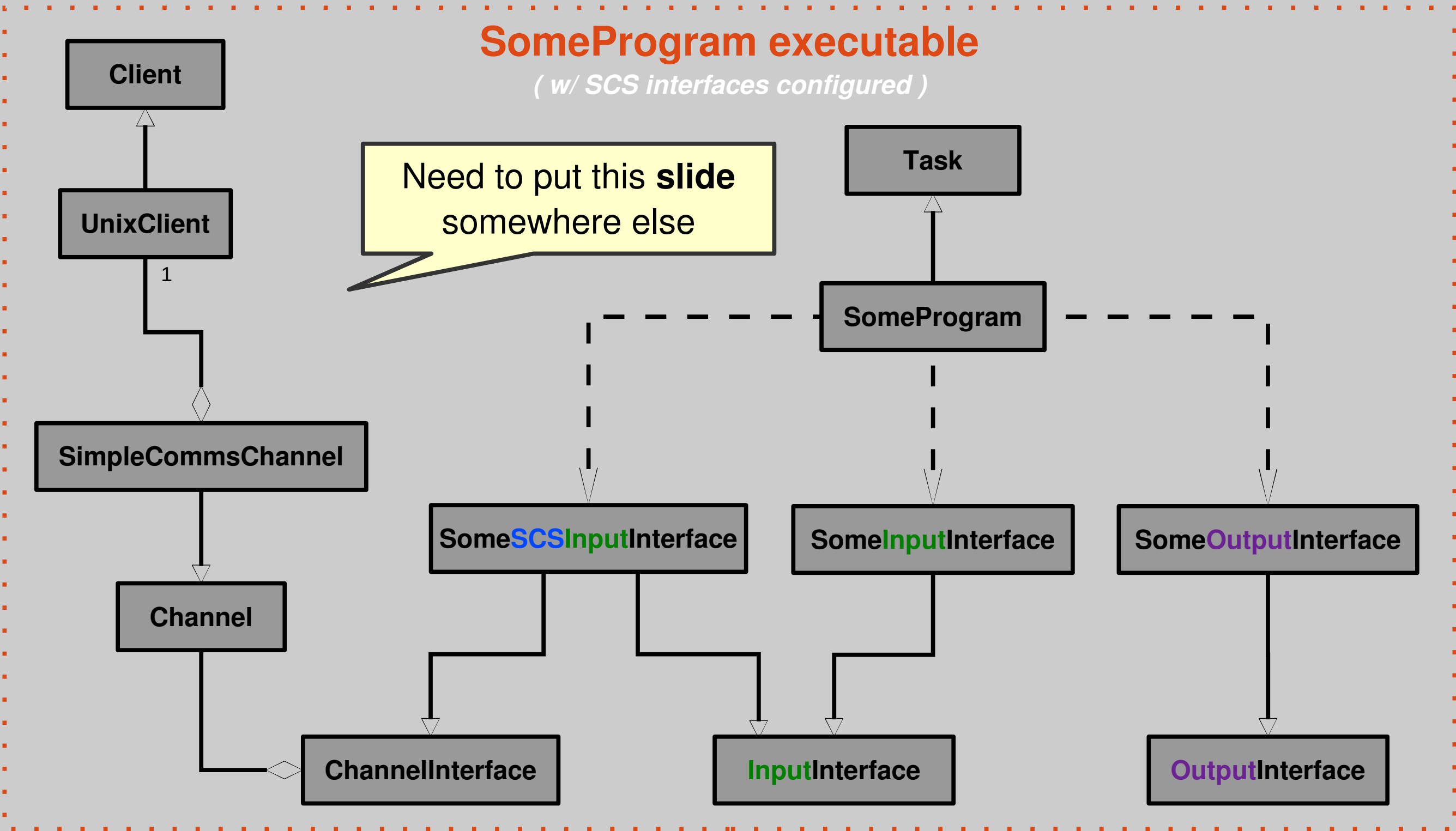
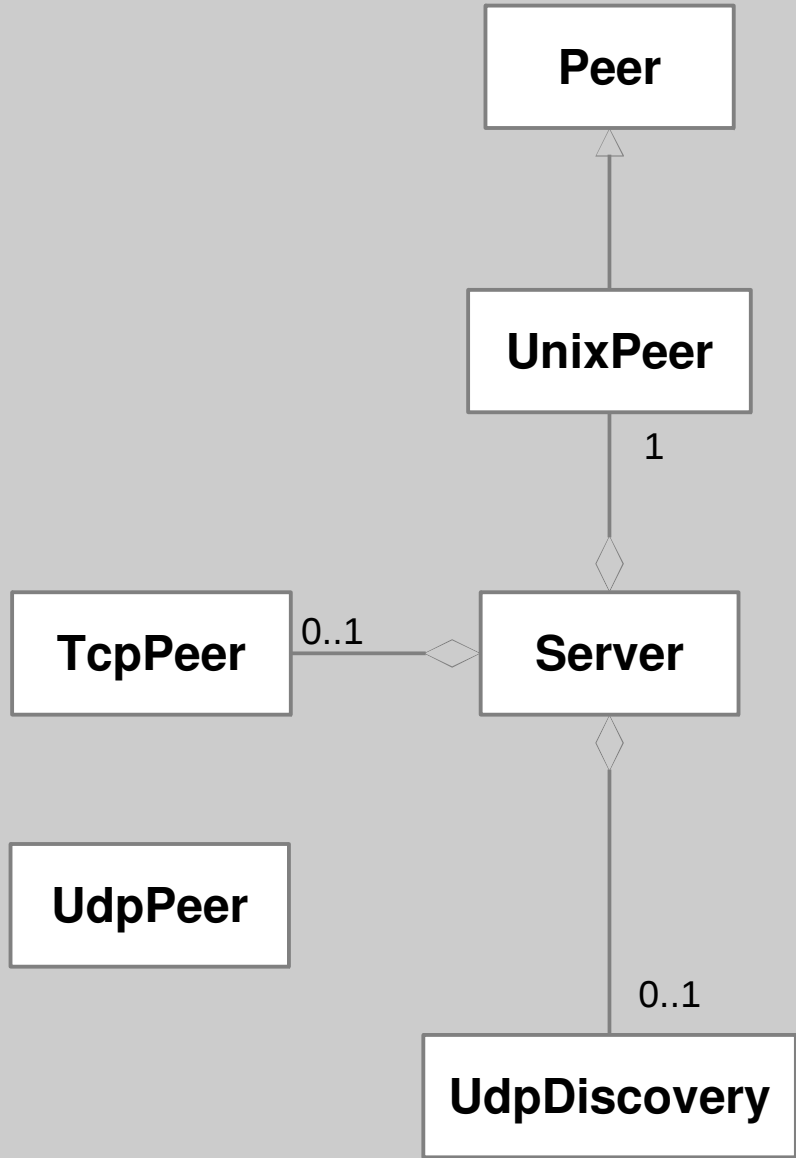
See AIS training part 3
i.e., this document



scs interface



Autonomy Task
w/**scs** interface(s)



class Interface

T - the datatype that flows
through the interface

```
template<class T> class Interface  
-----  
initialize( )  
cleanup( )  
getInterfaceName()
```

An interface shall have a name

ConfigSection

```
classDiagram  
    class Interface {  
        +T  
        +initialize()  
        +cleanup()  
        +getInterfaceName()  
    }  
    class ConfigSection  
    Interface o--> ConfigSection
```

class Interface - its configuration

```
template<class T> class Interface  
-----  
initialize( )  
cleanup( )  
getInterfaceName()
```

UML notation for **composition**

ConfigSection

In other words, each object of class **Interface** contains a data member to hold its configuration which is of class **ConfigSection**

class OutputInterface

template<class **T**> class **Interface**

initialize()

cleanup()

getInterfaceName()

ConfigSection



template <class **T**> class **OutputInterface**

publish(T &val)

OutputInterface is derived from
Interface and adds **publish** method

Example of an OutputInterface

File = prod/common/interfaces/Example/InterfaceTypes.h

```
typedef OutputInterface<ExampleType> ExampleOutput;
```

```
class TestTask: public Task
{
...
private:
    ExampleOutput *output_;
};
```

```
class TestTask: public Task
{
...
private:
    OutputInterface<ExampleType> *output_;
};
```

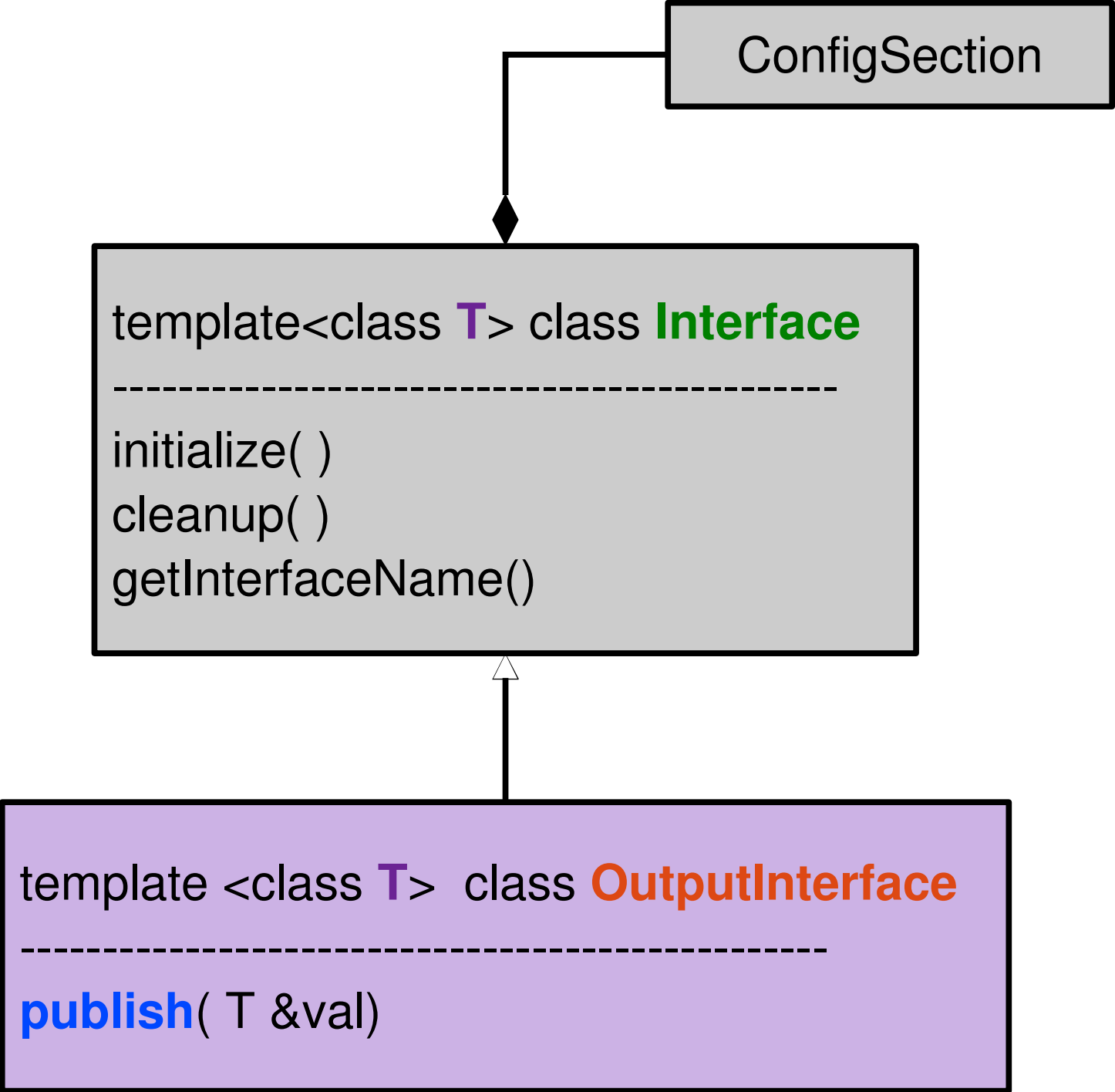
Just an alternative notation if the above **typedef** is not used

This syntax should be easier to follow than the “fancy” one.

publishing ExampleType

```
class TestTask: public Task
{
...
private:
    OutputInterface<ExampleType> *output_;
};
```

```
bool TestTask::executive( )
{
...
    static ExampleType    type;
    type.example++;
    output_->publish(
        type );
```



what does this publish do ?

```
class TestTask: public Task
{
...
private:
    OutputInterface<ExampleType> *output_;
};
```

```
bool TestTask::executive( )
{
...

```

```
output_->publish(
    type );
```

Question: What does this **publish** exactly do ?

Question: Does this **publish** transmit over SCS ?

Let's find out...

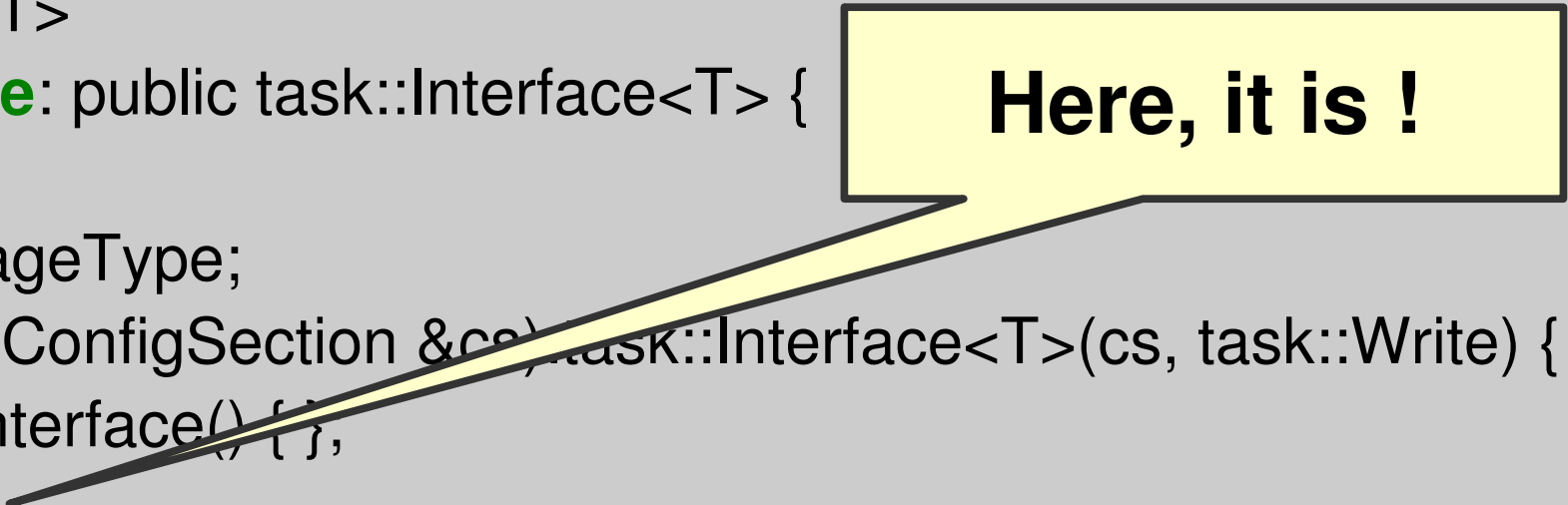
searching for publish implementation

The **publish** method is located in prod/common/interfaces/baseTypes/**OutputInterface.h**

```
template <typename T>
class OutputInterface: public task::Interface<T> {
public:
    typedef T MessageType;
    OutputInterface(ConfigSection &cs): task::Interface<T>(cs, task::Write) { };
    virtual ~OutputInterface() { },

    virtual bool publish(const T &val) = 0;

    virtual bool initialize( void ) = 0;
    virtual void cleanup( void ) = 0;
};
```



Here, it is !

pure function & abstract class

```
template <typename T>
class OutputInterface: public task::Interface<T> {
    public:
    .....

    virtual bool publish(const T &val) = 0;

};
```

- 1) There is no corresponding **OutputInterface.cpp** where **publish** could be implemented
- 2) Moreover, the syntax “virtual **method-name**(..) =0;” says specifically that **method-name**() does not have an implementation. Here “=0;” means no implementation. This method is called a **pure** virtual function.
- 3) A class with at least one **pure** virtual function is called an **abstract** class.
Any attempt to instantiate an object of such a class will result in a compiler error.
- 4) **Abstract** classes are used to derive other concrete classes.

Cannot work

```
class TestTask: public Task
{
...
private:
    OutputInterface<ExampleType> *output_;
};
```

```
bool TestTask::executive( )
{
...
    static ExampleType type;
    type.example++;
    output_->publish(
        type );
```

```
template<class T> class Interface
-----
initialize( )
cleanup( )
getInterfaceName()
```

ConfigSection

```
template <class T> class OutputInterface
-----
publish( T &val)
```

Therefore, this cannot work

This will work

```
class TestTask: public Task
{
...
private:
    OutputInterface<ExampleType> *output_;
};
```

```
bool TestTask::executive( )
{
...
    static ExampleType type;
    type.example++;
    output_->publish(type);
}
```

```
template<class T> class Interface
-----
...
```

```
template <class T> class OutputInterface
-----
publish( T &val) = 0;
```

```
template <class T> class
SomeConcreteOutputInterface
-----
publish( T &val) { write object T on the whiteboard }
```

This will work

Asking again

```
class TestTask: public Task
{
...
private:
    OutputInterface<ExampleType> *output_;
};
```

```
bool TestTask::executive( )
{
...

```

Asking the question again:

◆ What does this **publish** exactly do ?

```
output_->publish(
    type );
```

Depends on the interface object

```
template<class T> class Interface
```

```
template <class T> class  
CANOutputInterface  
-----  
publish( T &val) {  
write object T to CAN port 0 }
```

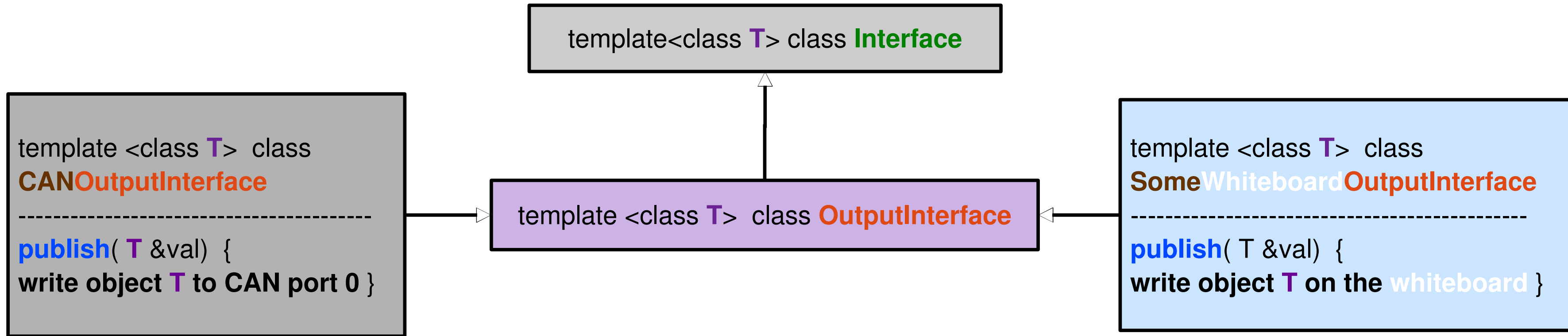
```
template <class T> class OutputInterface
```

```
template <class T> class  
SomeWhiteboardOutputInterface  
-----  
publish( T &val) {  
write object T on the whiteboard }
```

```
class TestTask: public Task  
{  
...  
private:  
    OutputInterface<ExampleType> *output_;
```

The answer totally depends which object
the **output_** points to.

Depends - cont'ed



- ◆ If **output_** points to an object of type **CANOutputInterface**, then the object is sent over the CAN bus.
- ◆ If **output_** points to an object of type **SomeWhiteboardOutputInterface**, then the object gets written on the **Whiteboard**.

Interface implementation

```
template<class T> class Interface
```

Interface implementation

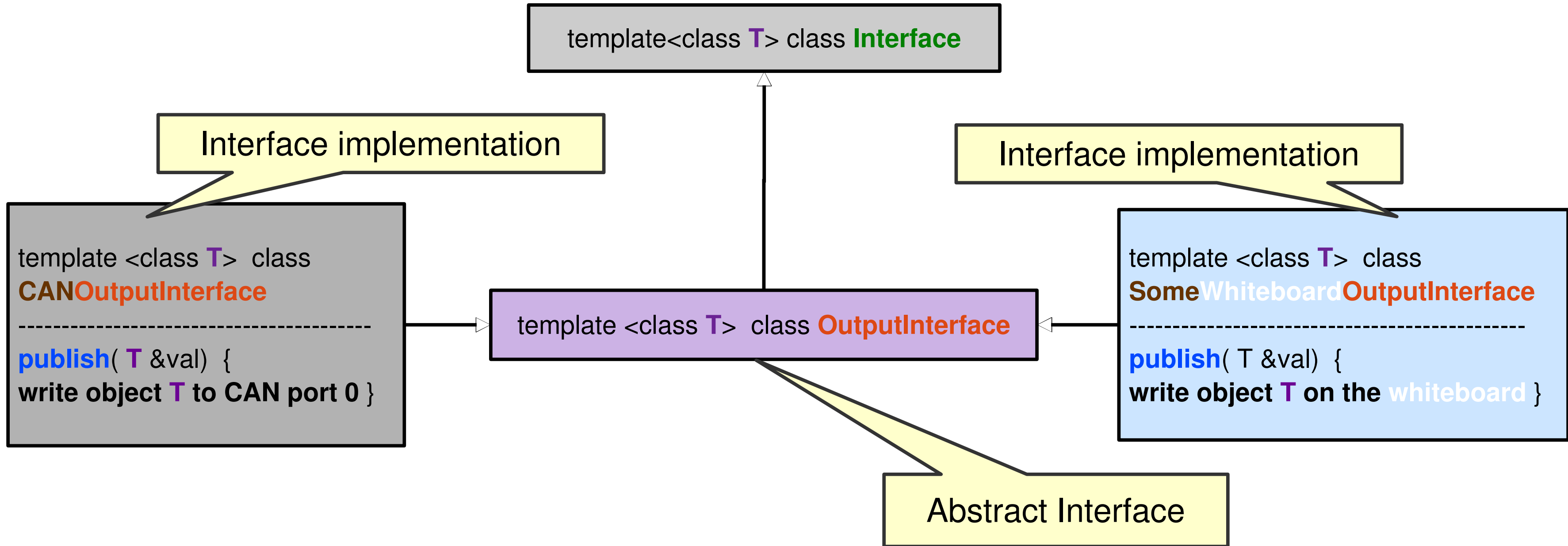
```
template <class T> class  
CANOutputInterface  
-----  
publish( T &val) {  
  write object T to CAN port 0 }
```

Interface implementation

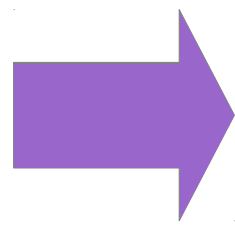
```
template <class T> class  
SomeWhiteboardOutputInterface  
-----  
publish( T &val) {  
  write object T on the whiteboard }
```

```
template <class T> class OutputInterface
```

Abstract Interface



Summarizing



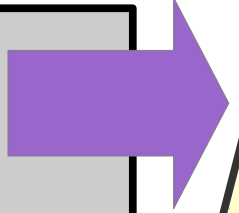
```
class TestTask: public Task
{
...
private:
    OutputInterface<ExampleType>
        *output_;
```

```
bool TestTask::executive( )
{
...
    static ExampleType type;
    type.example++;
    output_->publish( type );
```

- ◆ **TestTask** class declaration is not tied to any specific **OutputInterface**< **ExampleType**>.

Summarizing

```
class TestTask: public Task
{
...
private:
    OutputInterface<ExampleType>
        *output_;
```

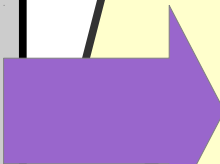


```
bool TestTask::executive( )
{
...
    static ExampleType type;
    type.example++;
    output_->publish( type );
```

- ◆ **TestTask** class declaration is not tied to any specific **OutputInterface**<**ExampleType**>.
- ◆ The declaration only specifies a pointer to some **abstract** **OutputInterface**<**ExampleType**> interface.

Summarizing

```
class TestTask: public Task
{
...
private:
    OutputInterface<ExampleType>
        *output_;
```



```
bool TestTask::executive( )
{
...
    static ExampleType type;
    type.example++;
    output_->publish( type );
```

- ◆ **TestTask** class declaration is not tied to any specific `OutputInterface<ExampleType>`.

- ◆ The declaration only specifies a pointer to some **abstract** `OutputInterface<ExampleType>` interface.

The **executive** method also does not care about the actual implementation for interface object (as long as the implementation is derived from `OutputInterface<ExampleType>` abstract interface).

Summarizing

```
class TestTask: public Task
{
...
private:
    OutputInterface<ExampleType>
        *output_;
```

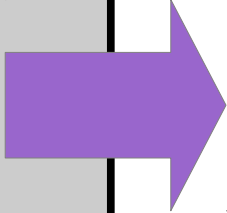
```
bool TestTask::executive( )
{
...
    static ExampleType type;
    type.example++;
    output_->publish( type );
```

- ◆ **TestTask** class declaration is not tied to any specific **OutputInterface**<**ExampleType**>.
- ◆ The declaration only specifies a pointer to some **abstract** **OutputInterface**<**ExampleType**> interface.
- ◆ The **executive** method also does not care about the actual implementation for interface object (as long as the implementation is derived from **OutputInterface**<**ExampleType**> abstract interface).
- ◆ This provides flexibility to specify what actual **OutputInterface** implementation should be used from some configuration file.

Summarizing

```
class TestTask: public Task
{
...
private:
    OutputInterface<ExampleType>
        *output_;
```

```
bool TestTask::executive( )
{
...
    static ExampleType type;
    type.example++;
    output_->publish( type );
```



- ◆ **TestTask** class declaration is not tied to any specific **OutputInterface**<**ExampleType**>.
- ◆ The declaration only specifies a pointer to some **abstract** **OutputInterface**<**ExampleType**> interface.
- ◆ The **executive** method also does not care about the actual implementation for interface object (as long as the implementation is derived from **OutputInterface**<**ExampleType**> abstract interface.
- ◆ This provides flexibility to specify what actual **OutputInterface** implementation should be used from some configuration file.
- ◆ The configuration file will specify what actual **OutputInterface**<**ExampleType**> implementation shall be used. Depending on its contents **ExampleType** will be displayed on the Whiteboard or sent over CAN datalink.

AIS configuration format

```
template<class T>
    class Interface
    -----
    initialize( )
    cleanup( )
    getInterfaceName()
```

ConfigSection



- ◆ AIS uses Ruby files to store configuration data.
- ◆ An autonomy task is given its configuration data using the following syntax:
./someAutonomyTask --instanceName=**someConfigurationFile**
- ◆ The above starts someAutonomyTask & **someConfigurationFile.rb** (located in **CAT_CONFIG_DIR** environment variable) is used to configure the Task.

TestWriteTask.rb

```
load "commonTaskParams.rb"
```

```
Interfaces = CommonTaskParams::Interfaces.dup;  
Interfaces.update( {  
  "ExampleOutput" => InterfaceDefs::ExampleOutput,  
} )
```

```
Parameters = CommonTaskParams::Parameters.dup;  
Parameters.update( {  
  "cycleRate_hz" => 50,  
} )
```

```
Execution = {  
  "executableName" => "testTask",  
}
```

This configuration section is to configure **interfaces** for TestTask

```
template<class T>  
  class Interface
```

```
-----  
  initialize( )  
  cleanup( )  
  getInterfaceName()
```

ConfigSection

TestWriteTask.rb

```
load "commonTaskParams.rb"

Interfaces = CommonTaskParams::Interfaces.dup;
Interfaces.update( {
  "ExampleOutput" => InterfaceDefs::ExampleOutput,
} )
```

This configuration section is to configure **interfaces** for TestTask

In other words, this section specifies what interface implementations should be used by the Task

ConfigSection & config

```
load "commonTaskParams.rb"

Interfaces = CommonTaskParams::Interfaces.dup;

Interfaces.update( {
  "ExampleOutput" => InterfaceDefs::ExampleOutput,
} )
```

Interface name

All configuration for **ExampleOutput** interface gets stored in here when interface implementation object gets instantiated.

ConfigSection

```
template<class T>
  class Interface
    -----
    initialize( )
    cleanup( )
    getInterfaceName()
```


TestWriteTask.rb interfaces

```
load "commonTaskParams.rb"
```

```
load "interfaces.rb"
```

```
Interfaces = {  
  "ProcessHealthOutput" => InterfaceDefs::ProcessHealthOutput,  
  "ProcessHealthInput"  => InterfaceDefs::ProcessHealthInput,  
  "RemoteLoggerTextOutput" => InterfaceDefs::ZRemoteLoggerTextOutput,  
  "AdjustLogLevelInput"   => InterfaceDefs::AdjustLogLevelInput,  
  "Clock"                 => InterfaceDefs::ClockSystem,  
}
```

```
Interfaces = CommonTaskParams::Interfaces.dup;  
Interfaces.update( {  
  "ExampleOutput" => InterfaceDefs::ExampleOutput,  
} )
```


interfaces.rb

This file contains all interface definitions that are available to an autonomyTask.

```
load "commonTaskParams.rb"
```

```
load "interfaces.rb"
```

```
Interfaces = {  
  "ProcessHealthOutput" => InterfaceDefs::ProcessHealthOutput,  
  "ProcessHealthInput"  => InterfaceDefs::ProcessHealthInput,  
  "RemoteLoggerTextOutput" => InterfaceDefs::ZRemoteLoggerTextOutput,  
  "AdjustLogLevelInput"   => InterfaceDefs::AdjustLogLevelInput,  
  "Clock"                 => InterfaceDefs::ClockSystem,  
}
```

```
Interfaces = CommonTaskParams::Interfaces.dup;  
Interfaces.update( {  
  "ExampleOutput" => InterfaceDefs::ExampleOutput,  
} )
```

default interfaces

```
load "commonTaskParams.rb"
```

```
load "interfaces.rb"
```

```
Interfaces = {  
  "ProcessHealthOutput" => InterfaceDefs::ProcessHealthOutput,  
  "ProcessHealthInput"  => InterfaceDefs::ProcessHealthInput,  
  "RemoteLoggerTextOutput" => InterfaceDefs::ZRemoteLoggerTextOutput,  
  "AdjustLogLevelInput"   => InterfaceDefs::AdjustLogLevelInput,  
  "Clock"                 => InterfaceDefs::ClockSystem,  
}
```

These are default interfaces that any
Autonomy task gets

interface insertion & addition

This inserts all the default interfaces

```
Interfaces = CommonTaskParams::Interfaces.dup;
```

This adds an interface called
ExampleOutput to testWriteTask

```
Interfaces.update( {  
  "ExampleOutput" => InterfaceDefs::ExampleOutput,  
} )
```

config data operations only

```
Interfaces = CommonTaskParams::Interfaces.dup;
```

```
Interfaces.update( {  
  "ExampleOutput" => InterfaceDefs::ExampleOutput,  
} )
```

- ◆ Remember that these are operations on the configuration data (within a text file.)
- ◆ These operations do not create any objects i.e., any interface objects

InterfaceDefs::ExampleOutput

ExampleOutput = {

```
"interface" => "Example.Channel.Output",  
# this describes the desired mechanism for transporting the raw payloads  
"channel" => {  
  "type" => "SimpleComms",      # most of our interfaces use the "SimpleComms" (aka "scs") transport  
  "queueSize" => 1,             # most channel types support some queuing of messages  
  "name" => "Example.Channel",  # this is the semantic name of the payload  
  "unixPath" => "scs",          # without an explicit reference to "unixPath", the default value will be scs  
}  
}
```

All these {key, value} pairs are stored in class **ConfigSection** when an interface implementation object gets created.

ConfigSection

```
template<class T>  
  class Interface
```

```
-----  
initialize( )  
cleanup( )  
getInterfaceName()
```

interface implementation shared library

Name of the shared library associated with the **interface implementation**

```
ExampleOutput = {  
  #  
  "interface" => "Example.Channel.Output",  
  # this describes the desired mechanism for transporting the raw payloads  
  "channel" => {  
    "type" => "SimpleComms",  
    "queueSize" => 1,  
    "name" => "Example.Channel",  
    "unixPath" => "scs",  
  }  
}
```

The shared library name is:
lib**Example.Channel.Output**.so

SimpleComms" (aka "scs") transport
queuing of messages
this is the semantic name of the payload
without an explicit reference to "unixPath", the default value will be scs

Interface database

Interface name

Sometime during Autonomy Task initialization

- read the **interfaces** section of the ruby config file
- for each interface
 - a) insert an entry into **InterfaceDb** (Interface Database)
 - b) load a shared library corresponding to that interface

```
ExampleOutput = {  
  "interface" => "Example.Channel.Output",  
  "channel" => {  
    "type" => "SimpleComms",  
    "queueSize" => 1,  
    "name" => "Example.Channel",  
  }  
}
```


Getting a pointer

Interface name

Sometime during Autonomy Task initialization

- read the interfaces section of the ruby config file
- for each interface
 - a) insert an entry into **InterfaceDb** (Interface Database)
 - b) load a shared library corresponding to that interface

```
ExampleOutput = {  
  "interface" => "Example.Channel.Output",  
  "channel" => {  
    "type" => "SimpleComms",  
    "queueSize" => 1,  
    "name" => "Example.Channel",  
  }  
}
```

Interface database for
Autonomy Task = TestTask

Interface name

```
bool TestTask::initialize( )
```

```
{  
  ➡ output_ = dynamic_cast<ExampleOutput*>( InterfaceDb::fetch( "ExampleOutput" ) );  
}
```


Getting a pointer - cont'd

Interface name

```
ExampleOutput = {  
  "interface" => "Example.Channel.Output",  
  "channel" => {  
    "type" => "SimpleComms",  
    "queueSize" => 1,  
    "name" => "Example.Channel",  
  }  
}
```

This gets a pointer to the
interface implementation

```
bool TestTask::initialize(  
{  
  output_ = dynamic_cast<ExampleOutput*>( InterfaceDb::fetch( "ExampleOutput" ) );  
}
```

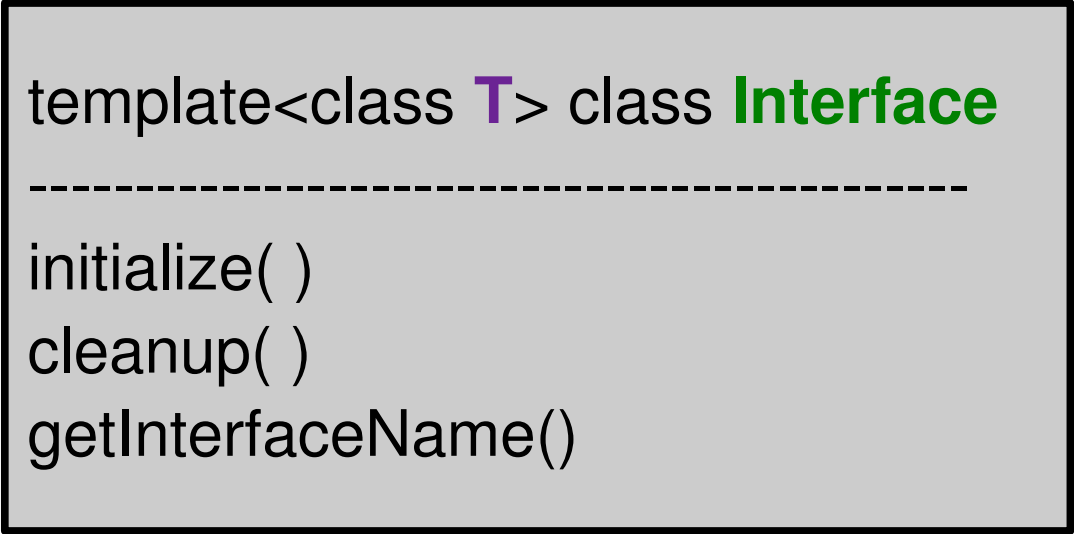
publishing

```
bool TestTask::executive( )  
{  
    static ExampleType some-var;  
    some-var.example++;  
    output_->publish( some-var );  
}
```

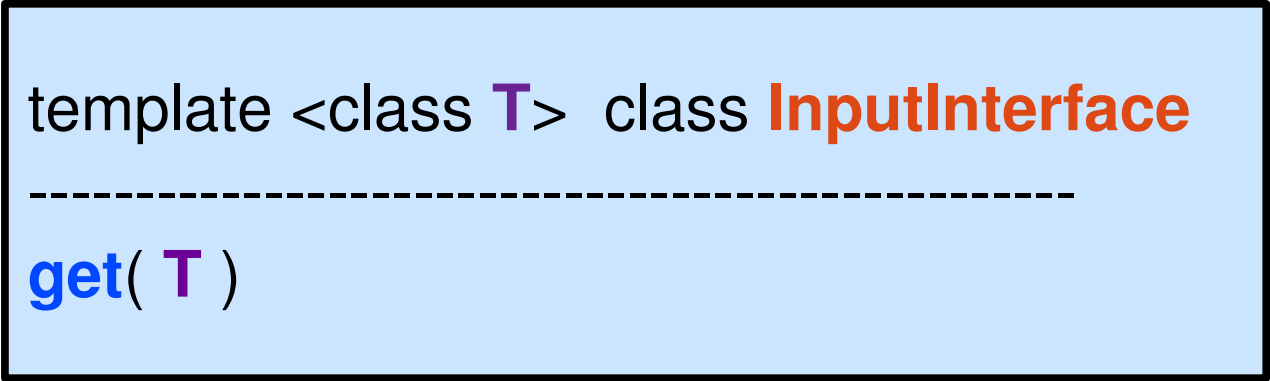
interface implementation packaging

- ◆ AIS build process packages Input and Output Interface implementations as shared libraries.
- ◆ During initialization, an Autonomy Task executable loads its interface implementations.
- ◆ Autonomy Tasks executable knows what interface implementations to load based on its interfaces configuration section.

class InputInterface



ConfigSection



Another example: keyswitch

Shutdown class encapsulates logic of putting an autonomy controller into a low power mode in a controlled and graceful fashion.

The logic includes but is not limited to:

- monitoring a number of signals to determine the right conditions to perform a shutdown
- notification of the imminent shutdown
- working together with some process manager to request a graceful process stoppage

The intention is that this class is included as part of some system or daemon task that is derived from class Task.

```
class Shutdown
{
    ...
    KeyswitchInput*    m_pKeySwitchInput;
    ...
};
```

keyswitch input

The keyswitch input is the most critical input monitored by the **Shutdown** class to determine whether the A5N6 should be shutdown (put into a low power mode).

```
class Shutdown
{
    ...
    KeyswitchInput*    m_pKeySwitchInput;
    ...
};
```

InputInterface<Keyswitch>

```
class Shutdown
```

```
{
```

```
    InputInterface<Keyswitch>* m_pKeySwitchInput;
```

```
},
```

The type that flows through the interface.

See file=prod/common/interfaces/Keyswitch/**InterfaceTypes.h**

```
class Shutdown
```

```
{
```

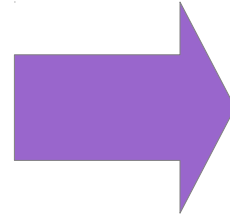
```
...
```

```
    KeyswitchInput* m_pKeySwitchInput;
```

```
...
```

```
};
```


InputInterface<Keyswitch>



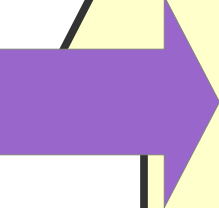
```
class Shutdown
{
    InputInterface<Keyswitch>*
    m_pKeySwitchInput;
};
```

- ◆ class **Shutdown** is not aware where the keyswitch input is coming from.

InputInterface<Keyswitch>

```
class Shutdown
{
    InputInterface<Keyswitch>*
    m_pKeySwitchInput;
};
```

◆ class **Shutdown** is not aware where the keyswitch input is coming from.



class **Shutdown** uses **m_pKeySwitchInput** which is a pointer to an abstract InputInterface.

InputInterface<Keyswitch>

```
class Shutdown
{
    InputInterface<Keyswitch>*
    m_pKeySwitchInput;
};
```

- ◆ class **Shutdown** is not aware where the keyswitch input is coming from.
 - ◆ class **Shutdown** uses **m_pKeySwitchInput** which is a pointer to an abstract InputInterface.
- It will be the configuration for PcmTask that specifies what is the actual interface implementation.

PcmCommon.rb

```
Interfaces.update( {  
  ...  
  "KeyswitchInput"    =>    InterfaceDefs::KeyswitchInputGPIO,  
  "KSOOutput"         =>    InterfaceDefs::KSOOutputGPIO,  
  ...  
} )
```